

Coffee in the Room

Código-fonte completo do sistema

19 de abril de 2026

Índice de arquivos

🗄️ BANCO DE DADOS

- lib/db/src/schema/products.ts
- lib/db/src/schema/clients.ts
- lib/db/src/schema/sales.ts
- lib/db/src/schema/orders.ts
- lib/db/src/schema/index.ts

🔌 BACKEND - API Server

- artifacts/api-server/src/app.ts
- artifacts/api-server/src/routes/index.ts
- artifacts/api-server/src/routes/products.ts
- artifacts/api-server/src/routes/clients.ts
- artifacts/api-server/src/routes/sales.ts
- artifacts/api-server/src/routes/admin.ts
- artifacts/api-server/src/routes/portal.ts
- artifacts/api-server/src/routes/orders.ts
- artifacts/api-server/src/middlewares/authMiddleware.ts

🖥️ FRONTEND

- artifacts/coffee-in-the-room/src/App.tsx
- artifacts/coffee-in-the-room/src/pages/Login.tsx
- artifacts/coffee-in-the-room/src/pages/DashboardLayout.tsx
- artifacts/coffee-in-the-room/src/pages/Products.tsx
- artifacts/coffee-in-the-room/src/pages/Clients.tsx
- artifacts/coffee-in-the-room/src/pages/Sales.tsx
- artifacts/coffee-in-the-room/src/pages/Admin.tsx
- artifacts/coffee-in-the-room/src/pages/ClientPortal.tsx
- artifacts/coffee-in-the-room/src/pages/ServiceOrders.tsx
- artifacts/coffee-in-the-room/src/hooks/useServiceOrders.ts

```
import { pgTable, serial, text, numeric, integer } from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";
import { z } from "zod/v4";

export const productsTable = pgTable("products", {
  id: serial("id").primaryKey(),
  userId: text("user_id").notNull(),
  name: text("name").notNull(),
  price: numeric("price", { precision: 10, scale: 2 }).notNull(),
  stock: integer("stock").notNull().default(0),
});

export const insertProductSchema = createInsertSchema(productsTable).omit({ id: true });
export type InsertProduct = z.infer<typeof insertProductSchema>;
export type Product = typeof productsTable.$inferSelect;
```

```
import { pgTable, serial, text, numeric, integer } from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";
import { z } from "zod/v4";

export const clientsTable = pgTable("clients", {
  id: serial("id").primaryKey(),
  userId: text("user_id").notNull(),
  name: text("name").notNull(),
  cpf: text("cpf").notNull(),
  phone: text("phone").notNull(),
  birthday: text("birthday"),
  code: integer("code").notNull(),
  totalPurchases: numeric("total_purchases", { precision: 10, scale:
2 }).notNull().default("0"),
  totalPaid: numeric("total_paid", { precision: 10, scale: 2 }).notNull().default("0"),
  balance: numeric("balance", { precision: 10, scale: 2 }).notNull().default("0"),
});

export const insertClientSchema = createInsertSchema(clientsTable).omit({ id: true });
export type InsertClient = z.infer<typeof insertClientSchema>;
export type Client = typeof clientsTable.$inferSelect;
```

```

import { pgTable, serial, text, timestamp, integer, numeric } from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";
import { z } from "zod/v4";

export const salesTable = pgTable("sales", {
  id: serial("id").primaryKey(),
  userId: text("user_id").notNull(),
  clientName: text("client_name").notNull(),
  productName: text("product_name").notNull(),
  paymentMethod: text("payment_method").notNull(),
  quantity: integer("quantity").notNull().default(1),
  installments: integer("installments").notNull().default(1),
  totalValue: numeric("total_value", { precision: 10, scale: 2 }).notNull().default("0"),
  installmentValue: numeric("installment_value", { precision: 10, scale:
2 }).notNull().default("0"),
  date: timestamp("date").notNull().defaultNow(),
});

export const insertSaleSchema = createInsertSchema(salesTable).omit({ id: true, date: true });
export type InsertSale = z.infer<typeof insertSaleSchema>;
export type Sale = typeof salesTable.$inferSelect;

```

```
import { pgTable, serial, text, numeric, integer, timestamp } from "drizzle-orm/pg-core";

export const ordersTable = pgTable("orders", {
  id: serial("id").primaryKey(),
  userId: text("user_id").notNull(),
  clientId: integer("client_id").notNull(),
  device: text("device").notNull(),
  problem: text("problem").notNull(),
  service: text("service").notNull(),
  price: numeric("price", { precision: 10, scale: 2 }).notNull(),
  status: text("status").notNull().default("pendente"),
  createdAt: timestamp("created_at").notNull().defaultNow(),
});

export type Order = typeof ordersTable.$inferSelect;
export type InsertOrder = typeof ordersTable.$inferInsert;
```

```
export * from "./products";  
export * from "./clients";  
export * from "./sales";  
export * from "./auth";  
export * from "./orders";
```

```
import express, { type Express } from "express";
import cors from "cors";
import cookieParser from "cookie-parser";
import path from "path";
import { fileURLToPath } from "url";
import { authMiddleware } from "../middlewares/authMiddleware";
import router from "../routes";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app: Express = express();

app.use(cors({ credentials: true, origin: true }));
app.use(cookieParser());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(authMiddleware);

app.use("/api", router);

app.use(express.static(path.join(__dirname, "../dist")));

app.get("/*path", (req, res) => {
  res.sendFile(path.join(__dirname, "../dist/index.html"));
});

export default app;
```

```
import { Router, type IRouter } from "express";
import healthRouter from "../health";
import authRouter from "../auth";
import portalRouter from "../portal";
import productsRouter from "../products";
import clientsRouter from "../clients";
import salesRouter from "../sales";
import adminRouter from "../admin";
import ordersRouter from "../orders";

const router: IRouter = Router();

router.use(healthRouter);
router.use(authRouter);
router.use(portalRouter);
router.use(productsRouter);
router.use(clientsRouter);
router.use(salesRouter);
router.use(adminRouter);
router.use(ordersRouter);

export default router;
```



```

import { Router, type IRouter } from "express";
import { db, productsTable } from "@workspace/db";
import { and, eq } from "drizzle-orm";
import {
  CreateProductBody,
  DeleteProductParams,
  QuickSellProductParams,
} from "@workspace/api-zod";

const router: IRouter = Router();

router.get("/products", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" }); return; }
  const products = await db
    .select()
    .from(productsTable)
    .where(eq(productsTable.userId, req.user.id))
    .orderBy(productsTable.id);
  res.json(products.map((p) => ({ id: p.id, name: p.name, price: parseFloat(p.price), stock: p.stock })));
});

router.post("/products", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" }); return; }
  const body = CreateProductBody.parse(req.body);
  const [product] = await db
    .insert(productsTable)
    .values({ userId: req.user.id, name: body.name, price: String(body.price), stock: body.stock })
    .returning();
  res.status(201).json({ id: product.id, name: product.name, price: parseFloat(product.price), stock: product.stock });
});

router.delete("/products/:id", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" }); return; }
  const { id } = DeleteProductParams.parse({ id: parseInt(req.params.id) });
  await db.delete(productsTable).where(and(eq(productsTable.id, id), eq(productsTable.userId, req.user.id)));
  res.json({ success: true });
});

router.post("/products/:id/sell", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" }); return; }
  const { id } = QuickSellProductParams.parse({ id: parseInt(req.params.id) });
  const [product] = await db
    .select()
    .from(productsTable)
    .where(and(eq(productsTable.id, id), eq(productsTable.userId, req.user.id)));
  if (!product) { res.status(404).json({ error: "Product not found" }); return; }
  if (product.stock <= 0) { res.status(400).json({ error: "Produto sem estoque" }); return; }
  const [updated] = await db
    .update(productsTable)
    .set({ stock: product.stock - 1 })
    .where(and(eq(productsTable.id, id), eq(productsTable.userId, req.user.id)))
    .returning();
  res.json({ id: updated.id, name: updated.name, price: parseFloat(updated.price), stock: updated.stock });
});

export default router;

```

```

import { Router, type IRouter } from "express";
import { db, clientsTable } from "@workspace/db";
import { and, eq } from "drizzle-orm";
import {
  CreateClientBody,
  DeleteClientParams,
  RegisterPurchaseParams,
  RegisterPurchaseBody,
  RegisterPaymentParams,
  RegisterPaymentBody,
} from "@workspace/api-zod";

const router: IRouter = Router();

function formatClient(c: typeof clientsTable.$inferSelect) {
  return {
    id: c.id,
    name: c.name,
    cpf: c.cpf,
    phone: c.phone,
    birthday: c.birthday ?? null,
    code: c.code,
    totalPurchases: parseFloat(c.totalPurchases),
    totalPaid: parseFloat(c.totalPaid),
    balance: parseFloat(c.balance),
  };
}

router.get("/clients", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" }); }
  return;
}
const clients = await db
  .select()
  .from(clientsTable)
  .where(eq(clientsTable.userId, req.user.id))
  .orderBy(clientsTable.id);
res.json(clients.map(formatClient));
});

router.post("/clients", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" }); }
  return;
}
const body = CreateClientBody.parse(req.body);

const [existing] = await db
  .select()
  .from(clientsTable)
  .where(and(eq(clientsTable.cpf, body.cpf), eq(clientsTable.userId, req.user.id)));

if (existing) {
  const addAmount = body.initialPurchase ?? 0;
  const newTotal = parseFloat(existing.totalPurchases) + addAmount;
  const newBalance = newTotal - parseFloat(existing.totalPaid);
  const [updated] = await db
    .update(clientsTable)
    .set({ name: body.name, phone: body.phone, birthday: body.birthday ?? null,
totalPurchases: String(newTotal), balance: String(newBalance) })
    .where(and(eq(clientsTable.cpf, body.cpf), eq(clientsTable.userId, req.user.id)))
    .returning();
  res.status(200).json(formatClient(updated));
  return;
}

const code = Math.floor(1000 + Math.random() * 9000);
const initialPurchase = body.initialPurchase ?? 0;
const [client] = await db
  .insert(clientsTable)
  .values({ userId: req.user.id, name: body.name, cpf: body.cpf, phone: body.phone, birthday: body.birthday ?? null, code, totalPurchases: String(initialPurchase), totalPaid: "0", balance: String(initialPurchase) })
  .returning();
res.status(201).json(formatClient(client));
});

router.delete("/clients/:id", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" }); }
  return;
}
const { id } = DeleteClientParams.parse({ id: parseInt(req.params.id) });
await db.delete(clientsTable).where(and(eq(clientsTable.id, id), eq(clientsTable.userId, req.user.id)));
res.json({ success: true });
});

```

```

router.post("/clients/:id/purchase", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" });
  return; }
  const { id } = RegisterPurchaseParams.parse({ id: parseInt(req.params.id) });
  const { amount } = RegisterPurchaseBody.parse(req.body);
  const [client] = await
db.select().from(clientsTable).where(and(eq(clientsTable.id, id),
eq(clientsTable.userId, req.user.id)));
  if (!client) { res.status(404).json({ error: "Client not found" }); return; }
  const newTotal = parseFloat(client.totalPurchases) + amount;
  const newBalance = newTotal - parseFloat(client.totalPaid);
  const [updated] = await db
    .update(clientsTable)
    .set({ totalPurchases: String(newTotal), balance: String(newBalance) })
    .where(and(eq(clientsTable.id, id), eq(clientsTable.userId, req.user.id)))
    .returning();
  res.json(formatClient(updated));
});

router.post("/clients/:id/payment", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" });
  return; }
  const { id } = RegisterPaymentParams.parse({ id: parseInt(req.params.id) });
  const { amount } = RegisterPaymentBody.parse(req.body);
  const [client] = await
db.select().from(clientsTable).where(and(eq(clientsTable.id, id),
eq(clientsTable.userId, req.user.id)));
  if (!client) { res.status(404).json({ error: "Client not found" }); return; }
  const newPaid = parseFloat(client.totalPaid) + amount;
  const newBalance = parseFloat(client.totalPurchases) - newPaid;
  const [updated] = await db
    .update(clientsTable)
    .set({ totalPaid: String(newPaid), balance: String(newBalance) })
    .where(and(eq(clientsTable.id, id), eq(clientsTable.userId, req.user.id)))
    .returning();
  res.json(formatClient(updated));
});

export default router;

```

```

import { Router, type IRouter } from "express";
import { db, salesTable, productsTable, clientsTable } from "@workspace/db";
import { and, eq } from "drizzle-orm";
import { CreateSaleBody } from "@workspace/api-zod";

const router: IRouter = Router();

function formatDate(d: Date) {
  return d.toLocaleString("pt-BR", {
    day: "2-digit", month: "2-digit", year: "numeric",
    hour: "2-digit", minute: "2-digit",
  });
}

router.get("/sales", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" }); }
  return;
}
const sales = await db
  .select()
  .from(salesTable)
  .where(eq(salesTable.userId, req.user.id))
  .orderBy(salesTable.id);
res.json(sales.map((s) => ({
  id: s.id,
  clientName: s.clientName,
  productName: s.productName,
  paymentMethod: s.paymentMethod,
  quantity: s.quantity,
  installments: s.installments,
  totalValue: parseFloat(s.totalValue),
  installmentValue: parseFloat(s.installmentValue),
  date: formatDate(s.date),
})));
});

router.post("/sales", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" }); }
  return;
}
const body = CreateSaleBody.parse(req.body);
const installments = Math.max(1, body.installments);
const uid = req.user.id;

const [product] = await db
  .select()
  .from(productsTable)
  .where(and(eq(productsTable.name, body.productName), eq(productsTable.userId, uid)));

if (product) {
  if (product.stock < body.quantity) { res.status(400).json({ error: "Estoque insuficiente" }); return; }
  await db
    .update(productsTable)
    .set({ stock: product.stock - body.quantity })
    .where(and(eq(productsTable.name, body.productName), eq(productsTable.userId, uid)));
}

const pricePerUnit = product ? parseFloat(product.price) : 0;
const totalValue = pricePerUnit * body.quantity;
const installmentValue = totalValue / installments;

if (body.clientName !== "Avulso" && body.clientName.trim() !== "") {
  const [client] = await db
    .select()
    .from(clientsTable)
    .where(and(eq(clientsTable.name, body.clientName), eq(clientsTable.userId, uid)));
  if (client) {
    const newTotal = parseFloat(client.totalPurchases) + totalValue;
    const newBalance = newTotal - parseFloat(client.totalPaid);
    await db
      .update(clientsTable)
      .set({ totalPurchases: String(newTotal), balance: String(newBalance) })
      .where(and(eq(clientsTable.id, client.id), eq(clientsTable.userId, uid)));
  }
}

const [sale] = await db
  .insert(salesTable)
  .values({
    userId: uid,
    clientName: body.clientName,
    productName: body.productName,
    paymentMethod: body.paymentMethod,

```

```
        quantity: body.quantity,
        installments,
        totalValue: String(totalValue),
        installmentValue: String(installmentValue),
    })
    .returning();

res.status(201).json({
    id: sale.id,
    clientName: sale.clientName,
    productName: sale.productName,
    paymentMethod: sale.paymentMethod,
    quantity: sale.quantity,
    installments: sale.installments,
    totalValue: parseFloat(sale.totalValue),
    installmentValue: parseFloat(sale.installmentValue),
    date: formatDate(sale.date),
});
});

export default router;
```

```
import { Router, type IRouter } from "express";
import { db, clientsTable, productsTable, salesTable } from "@workspace/db";
import { eq } from "drizzle-orm";

const router: IRouter = Router();

router.post("/admin/reset", async (req, res) => {
  if (!req.isAuthenticated()) { res.status(401).json({ error: "Unauthorized" });
  return; }
  await db.delete(salesTable).where(eq(salesTable.userId, req.user.id));
  await db.delete(clientsTable).where(eq(clientsTable.userId, req.user.id));
  await db.delete(productsTable).where(eq(productsTable.userId, req.user.id));
  res.json({ success: true });
});

export default router;
```

```
import { Router, type IRouter } from "express";
import { db, clientsTable } from "@workspace/db";
import { eq } from "drizzle-orm";

const router: IRouter = Router();

router.get("/portal/client/:code", async (req, res) => {
  const code = parseInt(req.params.code);
  if (isNaN(code)) {
    res.status(400).json({ error: "Código inválido" });
    return;
  }

  const [client] = await db
    .select()
    .from(clientsTable)
    .where(eq(clientsTable.code, code));

  if (!client) {
    res.status(404).json({ error: "Código não encontrado" });
    return;
  }

  res.json({
    id: client.id,
    name: client.name,
    cpf: client.cpf,
    birthday: client.birthday ?? null,
    code: client.code,
    totalPurchases: parseFloat(client.totalPurchases),
    totalPaid: parseFloat(client.totalPaid),
    balance: parseFloat(client.balance),
  });
});

export default router;
```

```

import { Router } from "express";
import { db, ordersTable } from "@workspace/db";
import { eq } from "drizzle-orm";

const router = Router();

router.post("/orders", async (req, res) => {
  if (!req.isAuthenticated()) {
    res.status(401).json({ error: "Unauthorized" });
    return;
  }

  const { clientId, device, problem, service, price } = req.body;

  const [order] = await db.insert(ordersTable).values({
    userId: req.user.id,
    clientId,
    device,
    problem,
    service,
    price: String(price),
  }).returning();

  res.status(201).json(order);
});

router.get("/orders", async (req, res) => {
  if (!req.isAuthenticated()) {
    res.status(401).json({ error: "Unauthorized" });
    return;
  }

  const orders = await db
    .select()
    .from(ordersTable)
    .where(eq(ordersTable.userId, req.user.id))
    .orderBy(ordersTable.id);

  res.json(orders);
});

router.patch("/orders/:id/status", async (req, res) => {
  if (!req.isAuthenticated()) {
    res.status(401).json({ error: "Unauthorized" });
    return;
  }

  const id = parseInt(req.params.id);
  const { status } = req.body;

  const [updated] = await db
    .update(ordersTable)
    .set({ status })
    .where(eq(ordersTable.id, id))
    .returning();

  res.json(updated);
});

router.delete("/orders/:id", async (req, res) => {
  if (!req.isAuthenticated()) {
    res.status(401).json({ error: "Unauthorized" });
    return;
  }

  const id = parseInt(req.params.id);
  await db.delete(ordersTable).where(eq(ordersTable.id, id));
  res.json({ success: true });
});

export default router;

```



```

import * as oidc from "openid-client";
import { type Request, type Response, type NextFunction } from
"express";
import type { AuthUser } from "@workspace/api-zod";
import {
  clearSession,
  getOidcConfig,
  getSessionId,
  getSession,
  updateSession,
  type SessionData,
} from "../lib/auth";

declare global {
  namespace Express {
    interface User extends AuthUser {}

    interface Request {
      isAuthenticated(): this is AuthedRequest;

      user?: User | undefined;
    }

    export interface AuthedRequest {
      user: User;
    }
  }
}

async function refreshIfExpired(
  sid: string,
  session: SessionData,
): Promise<SessionData | null> {
  const now = Math.floor(Date.now() / 1000);
  if (!session.expires_at || now <= session.expires_at) return
  session;

  if (!session.refresh_token) return null;

  try {
    const config = await getOidcConfig();
    const tokens = await oidc.refreshTokenGrant(
      config,
      session.refresh_token,
    );
    session.access_token = tokens.access_token;
    session.refresh_token = tokens.refresh_token ??
session.refresh_token;
    session.expires_at = tokens.expiresIn()
      ? now + tokens.expiresIn()!
      : session.expires_at;
    await updateSession(sid, session);
    return session;
  } catch {
    return null;
  }
}

export async function authMiddleware(
  req: Request,
  res: Response,
  next: NextFunction,
) {
  req.isAuthenticated = function (this: Request) {
    return this.user != null;
  } as Request["isAuthenticated"];

  const sid = getSessionId(req);
  if (!sid) {
    next();
    return;
  }

  const session = await getSession(sid);
  if (!session?.user?.id) {
    await clearSession(res, sid);
    next();
    return;
  }

  const refreshed = await refreshIfExpired(sid, session);
  if (!refreshed) {
    await clearSession(res, sid);
    next();
    return;
  }
}

```

```
}  
  req.user = refreshed.user;  
  next();  
}
```

```

import { Switch, Route, Router as WouterRouter, Redirect } from "wouter";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { Toaster } from "@components/ui/toaster";
import { TooltipProvider } from "@components/ui/tooltip";
import { useAuth } from "@workspace/replit-auth-web";

import Login from "../pages/Login";
import ClientPortal from "../pages/ClientPortal";
import Products from "../pages/Products";
import Clients from "../pages/Clients";
import Sales from "../pages/Sales";
import Admin from "../pages/Admin";
import ServiceOrders from "../pages/ServiceOrders";
import NotFound from "../pages/not-found";

const queryClient = new QueryClient();

function AuthenticatedRoutes() {
  const { isAuthenticated, isLoading } = useAuth();

  if (isLoading) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-background">
        <div className="w-10 h-10 border-2 border-primary border-t-transparent rounded-full animate-spin" />
      </div>
    );
  }

  if (!isAuthenticated) {
    return (
      <Switch>
        <Route path="/login" component={Login} />
        <Route path="/portal" component={ClientPortal} />
        <Route>{() => <Redirect to="/login" />}</Route>
      </Switch>
    );
  }

  return (
    <Switch>
      <Route path="/login">{() => <Redirect to="/products" />}</Route>
      <Route path="/portal" component={ClientPortal} />
      <Route path="/products" component={Products} />
      <Route path="/clients" component={Clients} />
      <Route path="/sales" component={Sales} />
      <Route path="/admin" component={Admin} />
      <Route path="/orders" component={ServiceOrders} />
      <Route path="/">{() => <Redirect to="/products" />}</Route>
      <Route path="/dashboard">{() => <Redirect to="/products" />}</Route>
      <Route component={NotFound} />
    </Switch>
  );
}

function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <TooltipProvider>
        <WouterRouter base={import.meta.env.BASE_URL.replace(/\/$/, "")}>
          <AuthenticatedRoutes />
        </WouterRouter>
      </TooltipProvider>
    </QueryClientProvider>
  );
}

export default App;

```

```

import { motion } from "framer-motion";
import { CoffeeScene } from "@components/CoffeeScene";
import { Button } from "@components/ui/button";
import { useAuth } from "@workspace/replit-auth-web";

export default function Login() {
  const { login } = useAuth();

  return (
    <div className="min-h-screen flex items-center justify-center p-4
relative overflow-hidden">
      { /* Dynamic Background */ }
      <div className="absolute inset-0 bg-[url('/images/coffee-bg.png')]
bg-cover bg-center opacity-40 mix-blend-overlay pointer-events-none" />
      <div className="absolute top-0 left-1/4 w-96 h-96 bg-primary/30
rounded-full mix-blend-screen filter blur-[100px] opacity-50 animate-
pulse" />
      <div className="absolute bottom-0 right-1/4 w-96 h-96 bg-accent/30
rounded-full mix-blend-screen filter blur-[100px] opacity-50 animate-
pulse delay-1000" />

      <motion.div
        initial={{ opacity: 0, y: 40 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.8, ease: "easeOut" }}
        className="relative z-10 w-full max-w-md"
      >
        <div className="bg-black/30 backdrop-blur-2xl border border-
white/10 rounded-3xl p-8 shadow-2xl">
          <CoffeeScene />

          <div className="text-center mb-8">
            <h1 className="text-3xl font-display font-bold text-white
mb-2">
              Bem-vindo ao
            <br />
            <span className="text-transparent bg-clip-text bg-gradient-
to-r from-primary to-accent">
              Coffee in the Room
            </span>
            </h1>
            <p className="text-white/60">Faça login para acessar o
sistema</p>
          </div>

          <Button onClick={login} className="w-full h-14 text-lg rounded-
xl">
            Entrar
          </Button>
        </div>
      </motion.div>
    </div>
  );
}

```

```

import { useEffect } from "react";
import { Link, useLocation } from "wouter";
import { Coffee, Users, ShoppingBag, Logout, Shield, Wrench }
  from "lucide-react";
import { cn } from "@lib/Utils";
import { useAuth } from "@workspace/replit-auth-web";
import { useListClients } from "@workspace/api-client-react";
import { useToast } from "@hooks/use-toast";

const navItems = [
  { name: "Produtos", href: "/products", icon: Coffee },
  { name: "Clientes", href: "/clients", icon: Users },
  { name: "Vendas", href: "/sales", icon: ShoppingBag },
  { name: "Ordens", href: "/orders", icon: Wrench },
  { name: "Admin", href: "/admin", icon: Shield },
];

export function DashboardLayout({ children }: { children:
  React.ReactNode }) {
  const [location] = useLocation();
  const { user, logout } = useAuth();
  const { data: clients } = useListClients();
  const { toast } = useToast();

  // Birthday alerts on mount
  useEffect(() => {
    if (!clients) return;
    const today = new Date();
    const mm = String(today.getMonth() + 1).padStart(2, "0");
    const dd = String(today.getDate()).padStart(2, "0");
    const todayStr = `${mm}-${dd}`;
    const birthdays = clients.filter(
      (c) => c.birthday && c.birthday.substring(5) === todayStr
    );
    if (birthdays.length > 0) {
      toast({
        title: "🎉 Aniversariantes de Hoje!",
        description: birthdays.map((b) => b.name).join(", "),
        duration: 10000,
      });
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [clients]);

  const displayName = [user?.firstName,
    user?.lastName].filter(Boolean).join(" ") || user?.email ||
    "Usuário";

  return (
    <div className="min-h-screen bg-background text-foreground
      flex flex-col md:flex-row">
      {/* Sidebar Desktop / Topbar Mobile */}
      <aside className="w-full md:w-64 border-b md:border-b-0
        md:border-r border-white/10 bg-black/20 backdrop-blur-xl p-4
        flex flex-col z-20">
        <div className="flex items-center gap-3 px-2 py-4
          mb-2">
          <div className="w-10 h-10 rounded-xl bg-gradient-to-
            br from-primary to-accent flex items-center justify-center
            shadow-lg shadow-primary/20 shrink-0">
            <Coffee className="w-6 h-6 text-white" />
          </div>
          <span className="font-display font-bold text-xl
            hidden md:block">
            Coffee Room
          </span>
        </div>

        {/* User greeting */}
        <div className="hidden md:block px-2 mb-6">
          <p className="text-xs text-muted-foreground">Bem-
            vindo,</p>
          <p className="text-sm font-semibold text-white
            truncate">{displayName}</p>
        </div>

        <nav className="flex md:flex-col gap-2 flex-1 overflow-
          x-auto md:overflow-visible no-scrollbar">
          {navItems.map((item) => {
            const isActive = location.startsWith(item.href);
            return (
              <Link key={item.href} href={item.href}>
                <div
                  className={cn(
                    "flex items-center gap-3 px-4 py-3 rounded-

```

```

xl transition-all duration-200 cursor-pointer whitespace-
nowrap",
        isActive
        ? "bg-primary/20 text-primary border
border-primary/30 shadow-inner"
        : "hover:bg-white/5 text-muted-
foreground hover:text-foreground"
    )}
    >
    <item.icon className={cn("w-5 h-5",
isActive ? "text-primary" : "")} />
    <span className="font-medium">{item.name}</
span>
  </div>
</Link>
);
}}}
</nav>

<button
  onClick={logout}
  className="mt-auto hidden md:flex items-center gap-3
px-4 py-3 text-muted-foreground hover:text-destructive
hover:bg-destructive/10 rounded-xl transition-colors"
  >
  <LogOut className="w-5 h-5" />
  <span className="font-medium">Sair</span>
</button>
</aside>

{ /* Main Content */}
<main className="flex-1 p-4 md:p-8 overflow-y-auto
relative z-10">
  <div className="absolute top-[-20%] right-[-10%] w-
[500px] h-[500px] bg-primary/20 rounded-full mix-blend-screen
filter blur-[120px] pointer-events-none" />
  <div className="relative z-10 max-w-6xl mx-
auto">{children}</div>
</main>
</div>
);
}

```

```

import { useState } from "react";
import { Plus, Package, Trash2, Zap } from "lucide-react";
import { useQueryClient } from "@tanstack/react-query";
import { DashboardLayout } from "../DashboardLayout";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Dialog, DialogContent, DialogHeader, DialogTitle } from "@components/ui/dialog";
import { useToast } from "@hooks/use-toast";
import { formatCurrency } from "@lib/utils";
import {
  useListProducts,
  useCreateProduct,
  useDeleteProduct,
  useQuickSellProduct,
  getListProductsQueryKey
} from "@workspace/api-client-react";

export default function Products() {
  const [isAddOpen, setIsAddOpen] = useState(false);
  const [formData, setFormData] = useState({ name: "", price: "", stock: "" });

  const { toast } = useToast();
  const queryClient = useQueryClient();

  const { data: products, isLoading } = useListProducts();

  const createProduct = useCreateProduct({
    mutation: {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey:
getListProductsQueryKey() });
        setIsAddOpen(false);
        setFormData({ name: "", price: "", stock: "" });
        toast({ title: "Produto cadastrado com sucesso!" });
      }
    }
  });

  const deleteProduct = useDeleteProduct({
    mutation: {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey:
getListProductsQueryKey() });
        toast({ title: "Produto removido!" });
      }
    }
  });

  const quickSell = useQuickSellProduct({
    mutation: {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey:
getListProductsQueryKey() });
        toast({ title: "Venda rápida registrada!" });
      }
    }
  });

  const handleCreate = (e: React.FormEvent) => {
    e.preventDefault();
    createProduct.mutate({
      data: {
        name: formData.name,
        price: parseFloat(formData.price),
        stock: parseInt(formData.stock, 10)
      }
    });
  };

  return (
    <DashboardLayout>
      <div className="flex justify-between items-center mb-8">
        <div>
          <h1 className="text-4xl font-display font-bold text-white mb-2">Produtos</h1>
          <p className="text-muted-foreground">Gerencie o estoque e cardápio</p>
        </div>
        <Button onClick={() => setIsAddOpen(true)}>
          <Plus className="w-5 h-5 mr-2" />
          Novo Produto
        </Button>
      </div>
    </DashboardLayout>
  );
}

```

```

        </Button>
      </div>

      {isLoading ? (
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3
gap-6">
          {[1,2,3].map(i => (
            <Card key={i} className="animate-pulse h-48 bg-white/5" />
          ))}
        </div>
      ) : (
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3
gap-6">
          {products?.map(product => (
            <Card key={product.id} className="group hover:border-
primary/50 transition-colors relative overflow-hidden">
              <div className="absolute top-0 right-0 p-4 opacity-10
group-hover:opacity-20 transition-opacity">
                <Package className="w-24 h-24 text-primary" />
              </div>
              <CardHeader>
                <CardTitle className="truncate pr-8">{product.name}</
CardTitle>
                <div className="text-3xl font-display font-bold text-
primary mt-2">
                  {formatCurrency(product.price)}
                </div>
              </CardHeader>
              <CardContent>
                <div className="flex justify-between items-center
mb-6">
                  <span className="text-sm text-muted-
foreground">Estoque disponível</span>
                  <span className={`px-3 py-1 rounded-full text-xs
font-bold ${product.stock > 0 ? 'bg-accent/20 text-accent' : 'bg-
destructive/20 text-destructive'}`}>
                    {product.stock} un
                  </span>
                </div>
                <div className="flex gap-2">
                  <Button
                    variant="secondary"
                    className="flex-1 bg-white/5 hover:bg-primary/20
hover:text-primary border border-white/10"
                    onClick={() => quickSell.mutate({ id:
product.id })}
                    disabled={product.stock <= 0 ||
quickSell.isPending}
                  >
                    <Zap className="w-4 h-4 mr-2" />
                    Venda Rápida
                  </Button>
                  <Button
                    variant="destructive"
                    size="icon"
                    onClick={() => {
                      if(confirm('Tem certeza que deseja excluir?')) {
                        deleteProduct.mutate({ id: product.id });
                      }
                    }}
                  >
                    <Trash2 className="w-4 h-4" />
                  </Button>
                </div>
              </CardContent>
            </Card>
          ))}
          {products?.length === 0 && (
            <div className="col-span-full py-12 text-center text-muted-
foreground">
              Nenhum produto cadastrado.
            </div>
          )}
        </div>
      )}

    <Dialog open={isAddOpen} onOpenChange={setIsAddOpen}>
      <DialogContent onClose={() => setIsAddOpen(false)}>
        <DialogHeader>
          <DialogTitle>Novo Produto</DialogTitle>
        </DialogHeader>
        <form onSubmit={handleCreate} className="space-y-4">
          <div className="space-y-2">
            <label className="text-sm font-medium text-
white/80">Nome</label>
            <Input
              required
              value={formData.name}

```



```

        onChange={e => setFormData({...formData, name:
e.target.value})}}
        placeholder="Ex: Cappuccino Duplo"
      />
    </div>
    <div className="grid grid-cols-2 gap-4">
      <div className="space-y-2">
        <label className="text-sm font-medium text-
white/80">Preço (R$)</label>
        <Input
          required
          type="number"
          step="0.01"
          min="0"
          value={formData.price}
          onChange={e => setFormData({...formData, price:
e.target.value})}}
          placeholder="0.00"
        />
      </div>
      <div className="space-y-2">
        <label className="text-sm font-medium text-
white/80">Estoque Inicial</label>
        <Input
          required
          type="number"
          min="0"
          value={formData.stock}
          onChange={e => setFormData({...formData, stock:
e.target.value})}}
          placeholder="0"
        />
      </div>
    </div>
    <Button
      type="submit"
      className="w-full mt-6"
      disabled={createProduct.isPending}
    >
      {createProduct.isPending ? "Salvando..." : "Salvar
Produto"}
    </Button>
  </form>
</DialogContent>
</Dialog>
</DashboardLayout>
);
}

```

```

import { useState } from "react";
import { Plus, User, Trash2, ArrowUpCircle, ArrowDownCircle, Search }
from "lucide-react";
import { useQueryClient } from "@tanstack/react-query";
import { DashboardLayout } from "../DashboardLayout";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Card, CardContent, CardHeader, CardTitle } from "@components/
ui/card";
import { Dialog, DialogContent, DialogHeader, DialogTitle } from "@/
components/ui/dialog";
import { useToast } from "@hooks/use-toast";
import { formatCurrency, maskCPF, maskPhone } from "@lib/utils";
import {
  useListClients,
  useCreateClient,
  useDeleteClient,
  useRegisterPurchase,
  useRegisterPayment,
  getListClientsQueryKey,
} from "@workspace/api-client-react";

export default function Clients() {
  const [isAddOpen, setIsAddOpen] = useState(false);
  const [search, setSearch] = useState("");
  const [formData, setFormData] = useState({
    name: "",
    cpf: "",
    phone: "",
    birthday: "",
    initialPurchase: "",
  });

  // Transaction Modal State
  const [transactionType, setTransactionType] = useState<"purchase" |
"payment" | null>(null);
  const [selectedClient, setSelectedClient] = useState<number |
null>(null);
  const [amount, setAmount] = useState("");

  const { toast } = useToast();
  const queryClient = useQueryClient();

  const { data: clients, isLoading } = useListClients();

  const createClient = useCreateClient({
    mutation: {
      onSuccess: (data) => {
        queryClient.invalidateQueries({ queryKey:
getListClientsQueryKey() });
        setIsAddOpen(false);
        setFormData({ name: "", cpf: "", phone: "", birthday: "",
initialPurchase: "" });
        // 200 = existing client updated, 201 = new client
        const msg =
          (data as { id: number }).id
            ? "Cliente salvo com sucesso!"
            : "Cliente cadastrado!";
        toast({ title: msg });
      },
    },
  });

  const deleteClient = useDeleteClient({
    mutation: {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey:
getListClientsQueryKey() });
        toast({ title: "Cliente removido!" });
      },
    },
  });

  const registerPurchase = useRegisterPurchase({
    mutation: {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey:
getListClientsQueryKey() });
        closeTransactionModal();
        toast({ title: "Compra registrada!" });
      },
    },
  });

  const registerPayment = useRegisterPayment({

```

```

    mutation: {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey:
getListClientsQueryKey() });
        closeTransactionModal();
        toast({ title: "Pagamento registrado!" });
      },
    },
  });

const handleCreate = (e: React.FormEvent) => {
  e.preventDefault();
  createClient.mutate({
    data: {
      name: formData.name,
      cpf: formData.cpf,
      phone: formData.phone,
      birthday: formData.birthday || null,
      initialPurchase: formData.initialPurchase ?
parseFloat(formData.initialPurchase) : null,
    },
  });
};

const handleTransaction = (e: React.FormEvent) => {
  e.preventDefault();
  if (!selectedClient || !amount) return;

  const val = parseFloat(amount);
  if (transactionType === "purchase") {
    registerPurchase.mutate({ id: selectedClient, data: { amount:
val } });
  } else {
    registerPayment.mutate({ id: selectedClient, data: { amount:
val } });
  }
};

const closeTransactionModal = () => {
  setTransactionType(null);
  setSelectedClient(null);
  setAmount("");
};

const filteredClients = clients?.filter(
(c) =>
  c.name.toLowerCase().includes(search.toLowerCase()) ||
c.cpf.includes(search)
);

return (
<DashboardLayout>
  <div className="flex flex-col md:flex-row justify-between items-
start md:items-center gap-4 mb-8">
    <div>
      <h1 className="text-4xl font-display font-bold text-white
mb-2">Clientes</h1>
      <p className="text-muted-foreground">Fidelidade e controle de
contas</p>
    </div>
    <div className="flex gap-4 w-full md:w-auto">
      <div className="relative flex-1 md:w-64">
        <Search className="absolute left-3 top-1/2 -translate-y-1/2
w-4 h-4 text-muted-foreground" />
        <Input
          placeholder="Buscar por nome ou CPF..."
          className="pl-9"
          value={search}
          onChange={(e) => setSearch(e.target.value)}
        />
      </div>
      <Button onClick={() => setIsAddOpen(true)}>
        <Plus className="w-5 h-5 mr-2" />
        <span className="hidden sm:inline">Adicionar / Atualizar</
span>
      </Button>
    </div>
  </div>

  {isLoading ? (
    <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
      {[1, 2, 3, 4].map((i) => (
        <Card key={i} className="animate-pulse h-64 bg-white/5" />
      ))}
    </div>
  ) : (
    <div className="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3
gap-6">

```

```

{filteredClients?.map((client) => (
  <Card
    key={client.id}
    className="group hover:border-primary/30 transition-all
duration-300"
  >
    <CardHeader className="pb-4 border-b border-white/5">
      <div className="flex justify-between items-start">
        <div>
          <CardTitle className="text-xl mb-1">{client.name}</
CardTitle>
          <p className="text-sm text-muted-foreground font-
mono">{maskCPF(client.cpf)}</p>
          {client.phone && (
            <p className="text-xs text-muted-foreground
mt-0.5">{maskPhone(client.phone)}</p>
          )}
        </div>
        <div className="bg-primary/20 text-primary px-3 py-1
rounded-lg text-sm font-bold border border-primary/20">
          Cód: {client.code}
        </div>
      </div>
    </CardHeader>
    <CardContent className="pt-4">
      <div className="grid grid-cols-2 gap-4 mb-6">
        <div className="bg-black/20 rounded-xl p-3 border
border-white/5">
          <p className="text-xs text-muted-foreground
mb-1">Total Comprado</p>
          <p className="font-bold text-
white">{formatCurrency(client.totalPurchases)}</p>
        </div>
        <div
          className={
            client.balance > 0
              ? "bg-destructive/10 rounded-xl p-3 border
border-destructive/20"
              : "bg-emerald-500/10 rounded-xl p-3 border
border-emerald-500/20"
          }
        >
          <p className="text-xs text-muted-foreground
mb-1">Saldo Devedor</p>
          <p
            className={`font-bold ${
              client.balance > 0 ? "text-destructive" : "text-
emerald-400"
            }`}
          >
            {formatCurrency(client.balance)}
          </p>
        </div>
      </div>

      <div className="flex gap-2">
        <Button
          variant="outline"
          className="flex-1 hover:bg-destructive/20 hover:text-
destructive hover:border-destructive/50"
          onClick={() => {
            setSelectedClient(client.id);
            setTransactionType("purchase");
          }}
        >
          <ArrowDownCircle className="w-4 h-4 mr-2" />
          Fiar
        </Button>
        <Button
          variant="outline"
          className="flex-1 hover:bg-emerald-500/20 hover:text-
emerald-400 hover:border-emerald-500/50"
          onClick={() => {
            setSelectedClient(client.id);
            setTransactionType("payment");
          }}
        >
          <ArrowUpCircle className="w-4 h-4 mr-2" />
          Pagar
        </Button>
        <Button
          variant="ghost"
          size="icon"
          className="text-muted-foreground hover:text-
destructive hover:bg-destructive/10"
          onClick={() => {
            if (confirm(`Excluir cliente ${client.name}? O
histórico será perdido.`)) {

```

```

        deleteClient.mutate({ id: client.id });
      }
    }}
  >
  <Trash2 className="w-4 h-4" />
</Button>
</div>
</CardContent>
</Card>
)}}
{filteredClients?.length === 0 && (
  <div className="col-span-full py-12 text-center text-muted-
foreground">
    Nenhum cliente encontrado.
  </div>
)}
</div>
)}
)}

{/* Add / Update Client Modal */}
<Dialog open={isAddOpen} onOpenChange={setIsAddOpen}>
  <DialogContent onClose={() => setIsAddOpen(false)}>
    <DialogHeader>
      <DialogTitle>
        <User className="w-5 h-5 inline mr-2 text-primary" />
        Adicionar / Atualizar Cliente
      </DialogTitle>
    </DialogHeader>
    <p className="text-xs text-muted-foreground -mt-2">
      Se o CPF já existir, o cadastro será atualizado.
    </p>
    <form onSubmit={handleCreate} className="space-y-4 mt-2">
      <div className="space-y-2">
        <label className="text-sm font-medium text-white/80">Nome
Completo</label>
        <Input
          required
          value={formData.name}
          onChange={(e) => setFormData({ ...formData, name:
e.target.value }}}
        />
      </div>
      <div className="space-y-2">
        <label className="text-sm font-medium text-white/80">CPF</
label>
        <Input
          required
          value={maskCPF(formData.cpf)}
          onChange={(e) => setFormData({ ...formData, cpf:
e.target.value.replace(/\D/g, "") }}}
          maxLength={14}
          placeholder="000.000.000-00"
        />
      </div>
      <div className="space-y-2">
        <label className="text-sm font-medium text-
white/80">Telefone</label>
        <Input
          required
          value={maskPhone(formData.phone)}
          onChange={(e) => setFormData({ ...formData, phone:
e.target.value.replace(/\D/g, "") }}}
          maxLength={15}
          placeholder="(00) 00000-0000"
        />
      </div>
      <div className="space-y-2">
        <label className="text-sm font-medium text-white/80">
          Data de Aniversário <span className="text-muted-
foreground">(opcional)</span>
        </label>
        <Input
          type="date"
          value={formData.birthday}
          onChange={(e) => setFormData({ ...formData, birthday:
e.target.value }}}
        />
      </div>
      <div className="space-y-2">
        <label className="text-sm font-medium text-white/80">
          Valor da Compra <span className="text-muted-
foreground">(opcional)</span>
        </label>
        <Input
          type="number"
          step="0.01"
          min="0"
          placeholder="R$ 0,00"
        />
      </div>
    </form>
  </DialogContent>
</Dialog>

```

```

        value={formData.initialPurchase}
        onChange={(e) => setFormData({ ...formData,
initialPurchase: e.target.value })}}
      />
      <p className="text-xs text-muted-foreground">
        Será adicionado ao saldo devedor do cliente.
      </p>
    </div>
    <Button type="submit" className="w-full mt-6"
disabled={createClient.isPending}>
      {createClient.isPending ? "Salvando..." : "Salvar Cliente"}
    </Button>
  </form>
</DialogContent>
</Dialog>

  { /* Transaction Modal */ }
  <Dialog
    open={transactionType !== null}
    onOpenChange={(open) => !open && closeTransactionModal()}
  >
    <DialogContent onClose={closeTransactionModal}>
      <DialogHeader>
        <DialogTitle
          className={
            transactionType === "purchase" ? "text-destructive" :
"text-emerald-400"
          }
        >
          {transactionType === "purchase"
            ? "Registrar Nova Compra (Fiado)"
            : "Registrar Pagamento"}
        </DialogTitle>
      </DialogHeader>
      <form onSubmit={handleTransaction} className="space-y-4 mt-4">
        <div className="space-y-2">
          <label className="text-sm font-medium text-white/80">Valor
(R$)</label>
          <Input
            required
            type="number"
            step="0.01"
            min="0.01"
            value={amount}
            onChange={(e) => setAmount(e.target.value)}
            placeholder="0.00"
            className="text-2xl h-14"
            autoFocus
          />
        </div>
        <Button
          type="submit"
          className={`w-full mt-6 h-14 text-lg ${
            transactionType === "purchase"
              ? "bg-destructive hover:bg-destructive/90"
              : "bg-emerald-500 hover:bg-emerald-600 text-white"
          }}
          disabled={registerPurchase.isPending ||
registerPayment.isPending}
        >
          Confirmar
        </Button>
      </form>
    </DialogContent>
  </Dialog>
</DashboardLayout>
);
}

```

```

import { useState } from "react";
import { ShoppingBag, CreditCard, Banknote, QRCode, Hash } from "lucide-react";
import { useQueryClient } from "@tanstack/react-query";
import { DashboardLayout } from "../DashboardLayout";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Card, CardContent } from "@components/ui/card";
import { useToast } from "@hooks/use-toast";
import { formatCurrency } from "@lib/Utils";
import {
  useListSales,
  useCreateSale,
  useListProducts,
  useListClients,
  getListSalesQueryKey,
  getListProductsQueryKey,
} from "@workspace/api-client-react";

export default function Sales() {
  const [formData, setFormData] = useState({
    clientName: "",
    productName: "",
    paymentMethod: "Dinheiro",
    quantity: 1,
    installments: 1,
  });

  const { toast } = useToast();
  const queryClient = useQueryClient();

  const { data: sales, isLoading: loadingSales } = useListSales();
  const { data: products } = useListProducts();
  const { data: clients } = useListClients();

  const createSale = useCreateSale({
    mutation: {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey:
getListSalesQueryKey() });
        queryClient.invalidateQueries({ queryKey:
getListProductsQueryKey() });
        setFormData({ clientName: "", productName: "", paymentMethod:
"Dinheiro", quantity: 1, installments: 1 });
        toast({ title: "Venda registrada com sucesso!" });
      },
      onError: (err: unknown) => {
        const msg = (err as { response?: { data?: { error?:
string } } })?.response?.data?.error;
        toast({ title: msg ?? "Erro ao registrar venda.", variant:
"destructive" });
      },
    },
  });

  const handleCreate = (e: React.FormEvent) => {
    e.preventDefault();
    createSale.mutate({ data: formData });
  };

  const selectedProduct = products?.find((p) => p.name ===
formData.productName);

  const getPaymentIcon = (method: string) => {
    switch (method) {
      case "Pix":
        return <QRCode className="w-4 h-4 text-emerald-400" />;
      case "Cartão":
        return <CreditCard className="w-4 h-4 text-blue-400" />;
      default:
        return <Banknote className="w-4 h-4 text-amber-400" />;
    }
  };

  return (
    <DashboardLayout>
      <div className="mb-8">
        <h1 className="text-4xl font-display font-bold text-white
mb-2">PDV & Vendas</h1>
        <p className="text-muted-foreground">Ponto de venda e histórico</p>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
        { /* Nova Venda Form */ }
        <div className="lg:col-span-1">

```

```

<Card className="sticky top-8 border-primary/30 shadow-
[0_0_30px_rgba(139,92,246,0.1)]">
  <div className="bg-gradient-to-r from-primary to-accent p-6
rounded-t-2xl">
    <h2 className="text-xl font-bold font-display flex items-
center gap-2">
      <ShoppingBag className="w-5 h-5" />
      Registrar Venda
    </h2>
  </div>
  <CardContent className="p-6">
    <form onSubmit={handleCreate} className="space-y-5">
      {/* Cliente */}
      <div className="space-y-2">
        <label className="text-sm font-medium text-
white/80">Cliente</label>
        <select
          required
          className="flex h-12 w-full rounded-xl border border-
white/10 bg-black/20 px-4 py-2 text-sm text-foreground shadow-inner
backdrop-blur-sm focus-visible:outline-none focus-visible:ring-2 focus-
visible:ring-primary focus-visible:border-primary appearance-none"
          value={formData.clientName}
          onChange={(e) => setFormData({ ...formData,
clientName: e.target.value })}
        >
          <option value="" disabled className="bg-[#2b0a3d]">
            Selecione um cliente
          </option>
          <option value="Avulso" className="bg-[#2b0a3d] font-
bold">
            Cliente Avulso
          </option>
          {clients?.map((c) => (
            <option key={c.id} value={c.name} className="bg-
[#2b0a3d]">
              {c.name} - {c.cpf}
            </option>
          ))}
        </select>
      </div>

      {/* Produto */}
      <div className="space-y-2">
        <label className="text-sm font-medium text-
white/80">Produto</label>
        <select
          required
          className="flex h-12 w-full rounded-xl border border-
white/10 bg-black/20 px-4 py-2 text-sm text-foreground shadow-inner
backdrop-blur-sm focus-visible:outline-none focus-visible:ring-2 focus-
visible:ring-primary focus-visible:border-primary appearance-none"
          value={formData.productName}
          onChange={(e) => setFormData({ ...formData,
productName: e.target.value })}
        >
          <option value="" disabled className="bg-[#2b0a3d]">
            Selecione um produto
          </option>
          <div>
            {products
              ?.filter((p) => p.stock > 0)
              .map((p) => (
                <option key={p.id} value={p.name} className="bg-
[#2b0a3d]">
                  {p.name} (R$ {p.price.toFixed(2)}) - estoque:
{p.stock}
                </option>
              ))}
            </div>
        </select>
      </div>

      {/* Quantidade */}
      <div className="space-y-2">
        <label className="text-sm font-medium text-white/80 flex
items-center gap-2">
          <Hash className="w-4 h-4" />
          Quantidade
          {selectedProduct && (
            <span className="text-xs text-muted-foreground ml-
auto">
              máx: {selectedProduct.stock}
            </span>
          )}
        </label>
        <Input
          type="number"
          min={1}
          max={selectedProduct?.stock ?? 999}

```



```

        required
        value={formData.quantity}
        onChange={(e) =>
            setFormData({ ...formData, quantity: Math.max(1,
                parseInt(e.target.value) || 1) })
        }
        className="text-center text-xl font-bold h-12"
    />
    { /* Subtotal preview */ }
    {selectedProduct && (
        <p className="text-right text-sm text-primary font-
semibold">
            Subtotal: R$ {(selectedProduct.price *
formData.quantity).toFixed(2)}
            {formData.installments > 1 && (
                <span className="text-white/50 font-normal ml-1">
                    ({formData.installments}x R$
                    {(selectedProduct.price * formData.quantity /
formData.installments).toFixed(2)})
                </span>
            )}
        </p>
    )}
    </div>

    { /* Forma de pagamento */ }
    <div className="space-y-2">
        <label className="text-sm font-medium text-
white/80">Forma de Pagamento</label>
        <div className="grid grid-cols-3 gap-2">
            {[ "Dinheiro", "Pix", "Cartão"].map((method) => (
                <button
                    key={method}
                    type="button"
                    onClick={() => setFormData({ ...formData,
paymentMethod: method })}
                    className={`flex flex-col items-center justify-
center p-3 rounded-xl border transition-all ${
                        formData.paymentMethod === method
                        ? "bg-primary/20 border-primary text-white"
                        : "bg-black/20 border-white/10 text-muted-
foreground hover:bg-white/5"
                    }`}
                >
                    <span className="mb-1">{getPaymentIcon(method)}</
span>
                    <span className="text-xs font-medium">{method}</
span>
                </button>
            )]}
        </div>
    </div>

    { /* Parcelas */ }
    <div className="space-y-2">
        <label className="text-sm font-medium text-white/80 flex
items-center gap-2">
            <CreditCard className="w-4 h-4" />
            Parcelas
        </label>
        <input
            type="number"
            min={1}
            required
            value={formData.installments}
            onChange={(e) =>
                setFormData({ ...formData, installments: Math.max(1,
                parseInt(e.target.value) || 1) })
            }
            className="text-center text-xl font-bold h-12"
        />
    </div>

    <button
        type="submit"
        className="w-full h-14 text-lg mt-4"
        disabled={createSale.isPending}
    >
        {createSale.isPending ? "Processando..." : "Finalizar
Venda"}
    </button>
</form>
</CardContent>
</Card>
</div>

{ /* Histórico */ }
<div className="lg:col-span-2 space-y-4">

```

```

    <h3 className="text-xl font-display font-bold text-white mb-4
px-2">
      Histórico Recente
    </h3>

    {loadingSales ? (
      <div className="space-y-3">
        {[1, 2, 3, 4].map((i) => (
          <Card key={i} className="animate-pulse h-20 bg-white/5" />
        ))}
      </div>
    ) : (
      <div className="space-y-3">
        {sales
          ?.slice()
          .reverse()
          .map((sale) => (
            <Card
              key={sale.id}
              className="bg-black/20 border-white/5 hover:bg-
black/40 transition-colors"
            >
              <CardContent className="p-4 flex items-center justify-
between">
                <div className="flex flex-col">
                  <span className="font-bold text-lg text-white">
                    {sale.productName}
                    {sale.quantity > 1 && (
                      <span className="ml-2 text-sm text-primary
font-normal">
                        ×{sale.quantity}
                      </span>
                    )}
                  </span>
                  <span className="text-sm text-muted-
foreground">{sale.clientName}</span>
                </div>
                <div className="flex flex-col items-end gap-1">
                  <span className="font-bold text-white text-base">
                    {formatCurrency(sale.totalValue)}
                  </span>
                  {sale.installments > 1 && (
                    <span className="text-xs text-primary font-
medium">
                      {sale.installments}x
                    </span>
                    {formatCurrency(sale.installmentValue)}
                  </span>
                </div>
                <div className="flex items-center gap-1 bg-white/5
px-2 py-1 rounded-md text-xs">
                  {getPaymentIcon(sale.paymentMethod)}
                  <span>{sale.paymentMethod}</span>
                </div>
                <span className="text-xs text-muted-
foreground">{sale.date}</span>
              </div>
            </CardContent>
          </Card>
        )}
      </div>
    )}
    {sales?.length === 0 && (
      <div className="py-12 text-center text-muted-foreground
border border-dashed border-white/10 rounded-2xl">
        Nenhuma venda registrada ainda.
      </div>
    )}
  </div>
</div>
</DashboardLayout>
);
}

```

```

import { useState } from "react";
import { Shield, KeyRound, Eye, EyeOff, CheckCircle2, AlertTriangle }
from "lucide-react";
import { useQueryClient } from "@tanstack/react-query";
import { DashboardLayout } from "../DashboardLayout";
import { Button } from "@components/ui/button";
import { Input } from "@components/ui/input";
import { Card, CardContent } from "@components/ui/card";
import { useToast } from "@hooks/use-toast";
import {
  useResetSystem,
  getListClientsQueryKey,
  getListProductsQueryKey,
  getListSalesQueryKey,
} from "@workspace/api-client-react";

const STORAGE_KEY = "systemCode";
const DEFAULT_CODE = "1234";

export function getSystemCode(): string {
  return localStorage.getItem(STORAGE_KEY) ?? DEFAULT_CODE;
}

export default function Admin() {
  const [currentCode, setCurrentCode] = useState("");
  const [newCode, setNewCode] = useState("");
  const [confirmCode, setConfirmCode] = useState("");
  const [showCurrent, setShowCurrent] = useState(false);
  const [showNew, setShowNew] = useState(false);
  const [changed, setChanged] = useState(false);

  const { toast } = useToast();
  const queryClient = useQueryClient();

  const resetSystem = useResetSystem({
    mutation: {
      onSuccess: () => {
        queryClient.invalidateQueries({ queryKey:
getListClientsQueryKey() });
        queryClient.invalidateQueries({ queryKey:
getListProductsQueryKey() });
        queryClient.invalidateQueries({ queryKey:
getListSalesQueryKey() });
      },
    },
  });

  const handleChangeCode = async (e: React.FormEvent) => {
    e.preventDefault();

    const stored = getSystemCode();
    if (currentCode !== stored) {
      toast({ title: "Código atual incorreto.", variant: "destructive" });
      return;
    }
    if (newCode.length < 4) {
      toast({ title: "O novo código deve ter pelo menos 4 caracteres.",
variant: "destructive" });
      return;
    }
    if (newCode !== confirmCode) {
      toast({ title: "Os novos códigos não coincidem.", variant:
"destructive" });
      return;
    }

    // Reset all system data first
    await resetSystem.mutateAsync({});

    // Save new code
    localStorage.setItem(STORAGE_KEY, newCode);
    setChanged(true);
    setCurrentCode("");
    setNewCode("");
    setConfirmCode("");
    toast({ title: "Código alterado e sistema zerado com sucesso!" });
  };

  return (
    <DashboardLayout>
      <div className="mb-8">
        <h1 className="text-4xl font-display font-bold text-white mb-2
flex items-center gap-3">
          <Shield className="w-8 h-8 text-primary" />
          Administração

```

```

    </h1>
    <p className="text-muted-foreground">Configurações e segurança do
sistema</p>
  </div>

  <div className="max-w-md">
    <Card className="border-primary/30 shadow-
[0_0_30px_rgba(139,92,246,0.1)]">
      <div className="bg-gradient-to-r from-primary to-accent p-6
rounded-t-2xl">
        <h2 className="text-xl font-bold font-display flex items-
center gap-2">
          <KeyRound className="w-5 h-5" />
          Mudar Código de Acesso
        </h2>
        <p className="text-sm text-white/70 mt-1">
          Altera o código e apaga todos os dados do sistema
        </p>
      </div>
      <CardContent className="p-6">
        {/* Warning banner */}
        <div className="mb-6 flex items-start gap-3 bg-amber-500/10
border border-amber-500/30 text-amber-400 rounded-xl p-4">
          <AlertTriangle className="w-5 h-5 flex-shrink-0 mt-0.5" />
          <p className="text-sm leading-relaxed">
            <span className="font-semibold">Atenção:</span> Ao mudar
o código, todos os clientes,
            produtos e vendas serão <span className="font-
semibold">apagados permanentemente</span>.
          </p>
        </div>

        {changed && (
          <div className="mb-6 flex items-center gap-2 bg-
emerald-500/10 border border-emerald-500/30 text-emerald-400 rounded-xl
p-4">
            <CheckCircle2 className="w-5 h-5 flex-shrink-0" />
            <span className="text-sm font-medium">
              Sistema zerado! Use o novo código no próximo login.
            </span>
          </div>
        )}

        <form onSubmit={handleChangeCode} className="space-y-5">
          <div className="space-y-2">
            <label className="text-sm font-medium text-
white/80">Código Atual</label>
            <div className="relative">
              <Input
                type={showCurrent ? "text" : "password"}
                required
                value={currentCode}
                onChange={(e) => setCurrentCode(e.target.value)}
                placeholder="Digite o código atual"
                className="pr-12"
              />
              <button
                type="button"
                onClick={() => setShowCurrent(!showCurrent)}
                className="absolute right-3 top-1/2 -translate-y-1/2
text-muted-foreground hover:text-white transition-colors"
              >
                {showCurrent ? <EyeOff className="w-4 h-4" /> : <Eye
className="w-4 h-4" />}
              </button>
            </div>
          </div>

          <div className="space-y-2">
            <label className="text-sm font-medium text-white/80">Novo
Código</label>
            <div className="relative">
              <Input
                type={showNew ? "text" : "password"}
                required
                minLength={4}
                value={newCode}
                onChange={(e) => setNewCode(e.target.value)}
                placeholder="Mínimo 4 caracteres"
                className="pr-12"
              />
              <button
                type="button"
                onClick={() => setShowNew(!showNew)}
                className="absolute right-3 top-1/2 -translate-y-1/2
text-muted-foreground hover:text-white transition-colors"
              >
                {showNew ? <EyeOff className="w-4 h-4" /> : <Eye

```

```

        className="w-4 h-4" />
      </button>
    </div>
  </div>

  <div className="space-y-2">
    <label className="text-sm font-medium text-white/80">Confirmar Novo Código</label>
    <input
      type="password"
      required
      value={confirmCode}
      onChange={(e) => setConfirmCode(e.target.value)}
      placeholder="Repita o novo código"
    />
  </div>

  <button
    type="submit"
    className="w-full h-12 mt-2 bg-destructive hover:bg-destructive/90"
    disabled={resetSystem.isPending}
  >
    {resetSystem.isPending ? "Zerando sistema..." : "Salvar Novo Código e Zerar Sistema"}
  </button>
</form>
</CardContent>
</Card>

<p className="text-xs text-muted-foreground text-center mt-4 px-4">
  O código padrão é <span className="font-mono text-primary">1234</span>. Guarde o novo código em local seguro – não é possível recuperá-lo sem o código atual.
</p>
</div>
</DashboardLayout>
);
}

```

```

import { useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { CoffeeScene } from "@components/CoffeeScene";
import { Input } from "@components/ui/input";
import { Button } from "@components/ui/button";
import { Card, CardContent } from "@components/ui/card";
import { formatCurrency } from "@lib/utils";
import { useGetClientByCode } from "@workspace/api-client-react";
import { User, ShoppingBag, Wallet, ArrowLeft } from "lucide-react";
import { Link } from "wouter";

export default function ClientPortal() {
  const [inputCode, setInputCode] = useState("");
  const [submittedCode, setSubmittedCode] = useState<number | null>(null);
  const [error, setError] = useState("");

  const { data: client, isLoading, isError } = useGetClientByCode(
    submittedCode ?? 0,
    { query: { enabled: submittedCode !== null } }
  );

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    const code = parseInt(inputCode);
    if (isNaN(code) || inputCode.trim() === "") {
      setError("Por favor, insira um código válido.");
      return;
    }
    setError("");
    setSubmittedCode(code);
  };

  const handleBack = () => {
    setSubmittedCode(null);
    setInputCode("");
    setError("");
  };

  const balance = client ? client.balance : 0;

  return (
    <div className="min-h-screen flex items-center justify-center p-4 relative overflow-hidden">
      <div className="absolute top-0 left-1/4 w-96 h-96 bg-primary/30 rounded-full mix-blend-screen filter blur-[100px] opacity-50 animate-pulse" />
      <div className="absolute bottom-0 right-1/4 w-96 h-96 bg-accent/30 rounded-full mix-blend-screen filter blur-[100px] opacity-50 animate-pulse delay-1000" />

      <motion.div
        initial={{ opacity: 0, y: 40 }}
        animate={{ opacity: 1, y: 0 }}
        transition={{ duration: 0.8, ease: "easeOut" }}
        className="relative z-10 w-full max-w-md"
      >
        <div className="bg-black/30 backdrop-blur-2xl border border-white/10 rounded-3xl p-8 shadow-2xl">
          <CoffeeScene />

          <AnimatePresence mode="wait">
            {!client ? (
              <motion.div
                key="login"
                initial={{ opacity: 0 }}
                animate={{ opacity: 1 }}
                exit={{ opacity: 0 }}
              >
                <div className="text-center mb-8">
                  <h1 className="text-2xl font-display font-bold text-white mb-2">
                    Área do Cliente
                  </h1>
                  <p className="text-white/60 text-sm">
                    Digite seu código de acesso
                  </p>
                </div>

                <form onSubmit={handleSubmit} className="space-y-4">
                  <Input
                    type="number"
                    placeholder="Código do cliente"
                    value={inputCode}
                    onChange={(e) => { setInputCode(e.target.value);
setError(""); }}

```

```

        className="text-center text-2xl tracking-[0.5em]
font-mono h-14"
        maxLength={4}
        autoFocus
    />
    {(error || (isError && submittedCode !== null)) &&
(
    <p className="text-destructive text-sm text-
center">
        {error || "Código não encontrado. Tente
novamente."}
    </p>
)}
    <Button
        type="submit"
        className="w-full h-12 text-base rounded-xl"
        disabled={isLoading}
    >
        {isLoading ? "Buscando..." : "Entrar"}
    </Button>
</form>

    <div className="mt-6 text-center">
        <Link href="/login">
            <span className="text-xs text-muted-foreground
hover:text-white transition-colors cursor-pointer flex items-center
justify-center gap-1">
                <ArrowLeft className="w-3 h-3" /> Área
administrativa
            </span>
        </Link>
    </div>
</motion.div>
) : (
    <motion.div
        key="profile"
        initial={{ opacity: 0, y: 10 }}
        animate={{ opacity: 1, y: 0 }}
        exit={{ opacity: 0 }}
        className="space-y-4"
    >
        <div className="text-center mb-6">
            <div className="w-16 h-16 rounded-full bg-gradient-
to-br from-primary to-accent flex items-center justify-center mx-
auto mb-3 shadow-lg">
                <User className="w-8 h-8 text-white" />
            </div>
            <h2 className="text-2xl font-display font-bold
text-white">
                Olá, {client.name.split(" ")[0]}!
            </h2>
            <p className="text-white/50 text-sm">Código
#{client.code}</p>
        </div>

        <div className="grid grid-cols-2 gap-3">
            <Card className="bg-black/20 border-white/5">
                <CardContent className="p-4 flex flex-col items-
center gap-1">
                    <ShoppingBag className="w-5 h-5 text-primary
mb-1" />
                    <p className="text-xs text-muted-
foreground">Total Compras</p>
                    <p className="font-bold text-white text-lg">
                        {formatCurrency(client.totalPurchases)}
                    </p>
                </CardContent>
            </Card>

            <Card className="bg-black/20 border-white/5">
                <CardContent className="p-4 flex flex-col items-
center gap-1">
                    <Wallet className="w-5 h-5 text-primary mb-1" /
                >
                    <p className="text-xs text-muted-
foreground">Total Pago</p>
                    <p className="font-bold text-white text-lg">
                        {formatCurrency(client.totalPaid)}
                    </p>
                </CardContent>
            </Card>
        </div>

        <Card className={`border ${balance > 0 ? "bg-
destructive/10 border-destructive/30" : "bg-green-500/10 border-
green-500/30"}`>
            <CardContent className="p-4 text-center">
                <p className="text-sm text-muted-foreground

```

```

mb-1">Saldo em Aberto</p>
      <p className={`text-3xl font-bold ${balance >
0 ? "text-destructive" : "text-green-400"}`}>
        {formatCurrency(balance)}
      </p>
      {balance <= 0 && (
        <p className="text-xs text-green-400/70
mt-1">Tudo em dia! ' </p>
      )}
    </CardContent>
  </Card>

  <Button
    variant="outline"
    className="w-full"
    onClick={handleBack}
  >
    <ArrowLeft className="w-4 h-4 mr-2" /> Sair
  </Button>
</motion.div>
  )}
</AnimatePresence>
</div>
</motion.div>
</div>
  );
}

```



```

import { useState } from "react";

type Order = {
  client: string;
  device: string;
  problem: string;
  price: number;
  status: string;
};

export default function ServiceOrders() {
  const [orders, setOrders] = useState<Order[]>([]);
  const [form, setForm] = useState<Order>({
    client: "",
    device: "",
    problem: "",
    price: 0,
    status: "Em análise",
  });

  function addOrder() {
    setOrders([...orders, form]);
    setForm({
      client: "",
      device: "",
      problem: "",
      price: 0,
      status: "Em análise",
    });
  }

  function updateStatus(index: number, status: string) {
    const newOrders = [...orders];
    newOrders[index].status = status;
    setOrders(newOrders);
  }

  return (
    <div className="p-4">
      <h1 className="text-xl font-bold mb-4">Ordens de Serviço</h1>

      <div className="grid gap-2 mb-4">
        <input placeholder="Cliente" value={form.client}
          onChange={(e) => setForm({ ...form, client:
e.target.value }}} />

        <input placeholder="Aparelho" value={form.device}
          onChange={(e) => setForm({ ...form, device:
e.target.value }}} />

        <input placeholder="Problema" value={form.problem}
          onChange={(e) => setForm({ ...form, problem:
e.target.value }}} />

        <input type="number" placeholder="Valor"
          onChange={(e) => setForm({ ...form, price:
Number(e.target.value) }}} />

        <button onClick={addOrder} className="bg-green-500 text-
white p-2">
          Criar OS
        </button>
      </div>

      {orders.map((o, i) => (
        <div key={i} className="border p-2 mb-2">
          <p><b>{o.client}</b> - {o.device}</p>
          <p>{o.problem}</p>
          <p>R$ {o.price}</p>
          <p>Status: {o.status}</p>

          <select onChange={(e) => updateStatus(i,
e.target.value)}>
            <option>Em análise</option>
            <option>Em manutenção</option>
            <option>Pronto</option>
          </select>
        </div>
      ))}
    </div>
  );
}

```

```

import { useQuery, useMutation, useQueryClient } from
"@tanstack/react-query";

export type OrderStatus = "pendente" | "em andamento" |
"concluído" | "cancelado";

export interface Order {
  id: number;
  userId: string;
  clientId: number;
  device: string;
  problem: string;
  service: string;
  price: string;
  status: OrderStatus;
  createdAt: string;
}

export interface CreateOrderInput {
  clientId: number;
  device: string;
  problem: string;
  service: string;
  price: number;
}

const QUERY_KEY = ["/api/orders"];

async function apiFetch(url: string, options?: RequestInit) {
  const res = await fetch(url, { credentials:
"include", ...options });
  if (!res.ok) {
    const data = await res.json().catch(() => ({}));
    throw new Error(data?.error ?? `Erro ${res.status}`);
  }
  return res.json();
}

export function useOrders() {
  return useQuery<Order[]>({
    queryKey: QUERY_KEY,
    queryFn: () => apiFetch("/api/orders"),
  });
}

export function useCreateOrder() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (data: CreateOrderInput) =>
      apiFetch("/api/orders", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(data),
      }),
    onSuccess: () => queryClient.invalidateQueries({ queryKey:
QUERY_KEY }),
  });
}

export function useUpdateOrderStatus() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: ({ id, status }: { id: number; status:
OrderStatus }) =>
      apiFetch(`/api/orders/${id}/status`, {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ status }),
      }),
    onSuccess: () => queryClient.invalidateQueries({ queryKey:
QUERY_KEY }),
  });
}

export function useDeleteOrder() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (id: number) =>
      apiFetch(`/api/orders/${id}`, { method: "DELETE" }),
    onSuccess: () => queryClient.invalidateQueries({ queryKey:
QUERY_KEY }),
  });
}

```